



Khoa Khoa Học Máy Tính

Nguyễn Hải Phong
Mã số sinh viên: **25521383**

Giảng viên phụ trách: **Nguyễn Thanh Sơn**

IT003.Q21.CTTN

[Game : Cycle of Curse]

May 2026

MỤC LỤC	i
1 Giới thiệu đồ án	1
1.1 Mô tả chung về Game	1
1.2 Các CTDL và giải thuật đã được sử dụng	1
1.2.1 Nhóm giải thuật tìm đường và Trí tuệ nhân tạo (AI)	1
1.2.2 Nhóm giải thuật Đồ họa và Hiển thị	1
1.2.3 Nhóm Cấu trúc dữ liệu (Data Structures)	2
2 Quá trình thực hiện	2
2.1 Tuần 1	2
2.1.1 Giai đoạn 1: Thiết lập nền tảng và Tiền xử lý đồ họa	2
2.1.2 Giai đoạn 2: Xây dựng cơ chế lõi và Quản lý không gian	3
2.1.3 Danh sách hình ảnh nhân vật và map và vũ khí	3
2.2 Tuần 2	7
2.2.1 Hệ thống chuyển động và Hiệu ứng hoạt ảnh	7
2.2.2 Cơ chế chiến đấu và Hiệu ứng thuật thức	7
2.2.3 Cơ chế xây dựng bản đồ và Tương tác Hitbox	7
2.3 Tuần 3: Hệ thống chỉ số, AI nâng cao và Hoàn thiện vòng lặp Gameplay	
Round 1 2	8
2.3.1 Hệ thống dữ liệu và Hiển thị nội suy (Juice & Feedback)	8
2.3.2 Hoàn thiện Round 1: Hệ thống AI và Cơ chế Boss Băng	8
2.3.3 Hoàn thiện Round 2: Boss Megumi và Thuật thức Triệu hồi	9
2.3.4 Cơ chế Combat và Hiệu ứng Trạng thái chuyên sâu	9
2.3.5 Quản lý Màn chơi và Tiền xử lý đồ họa tự động	9
2.3.6 Danh sách hình ảnh minh họa	10
2.4 Tuần 4: Hoàn thiện Boss Sukuna, Hệ thống Skill Tree và Menu khởi đầu	11
2.4.1 Hệ thống Boss Sukuna và Round cuối (Final Round)	11
2.4.2 Hệ thống Menu và Cây kỹ năng (Skill Tree)	11
2.4.3 Hệ thống Lưu trữ kỷ lục (Leaderboard System)	11
2.4.4 Tổng kết và Đóng gói sản phẩm	12
2.4.5 Danh sách hình ảnh minh họa	12
3 Kết quả đạt được	13
3.1 Về mặt Kỹ thuật và Giải thuật (Technical & Algorithms)	13
3.2 Về mặt Gameplay và Tính năng (Gameplay Features)	13
3.3 Về mặt Trải nghiệm và Dữ liệu (UX & Data)	13
4 Tài liệu tham khảo	14
A Phụ lục 1: Giới thiệu (demo) kết quả	14
B Phụ lục 2: Docstring	14
B.1 main.py — Entry Point & Game Loop	14
B.2 settings.py — Cấu hình Trung tâm	15
B.3 support.py — Hàm Tiện ích	15
B.4 tile.py — Ô Bản đồ Tĩnh	15
B.5 entity.py — Lớp Cơ sở Entity	16
B.6 weapon.py — Sprite Vũ khí	16

B.7	<code>magic.py</code> — Hệ thống Phép thuật	17
B.8	<code>particles.py</code> — Hiệu ứng Hạt	17
B.9	<code>menu.py</code> — Màn hình Menu	18
B.10	<code>ui.py</code> — Giao diện HUD	19
B.11	<code>level.py</code> — Điều phối Game World	19
B.12	<code>player.py</code> — Nhân vật Người chơi	20
B.13	<code>enemy.py</code> — AI Kẻ thù	21
B.14	<code>summon.py</code> — Shikigami (Linh thú)	22
B.15	Tổng kết DSA trong Project	24

1 Giới thiệu đề án

1.1 Mô tả chung về Game

Đề án tập trung xây dựng một trò chơi hành động nhập vai (Action RPG) thuộc thể loại **Boss Rush**. Trò chơi được lấy cảm hứng từ lối chơi linh hoạt của *Soul Knight* kết hợp với bối cảnh thế giới chú thuật đặc trưng của bộ truyện *Jujutsu Kaisen*.

Các đặc điểm nổi bật của ứng dụng bao gồm:

- **Cốt truyện và bối cảnh:** Game lấy ý tưởng từ bộ manga Jujutsu Kaisen. Người chơi nhập vai một con khỉ không có chú lực bị giam cầm trong một Mê cung huyền bí. Để phá vỡ kết giới và tìm đường thoát thân, người chơi buộc phải đối đầu và tiêu diệt các Boss của mỗi Round sở hữu sức mạnh áp đảo.
- **Cơ chế chiến đấu:** Bởi vì không có chú lực, Player sẽ sử dụng các *Chú cụ* được khảm trong nó các thuật thức đặc biệt. Player sẽ sử dụng các vũ khí, kỹ năng bộ trợ, và các vật phẩm từ game để tiến hành phá đảo 3 Round quái trong Game. Game cũng có cơ chế *Cây kỹ năng* từ đó giúp người dùng xây dựng các nâng cấp khác nhau tạo nên sự sáng tạo và phong cách riêng cho từng người chơi.
- **Trải nghiệm hình ảnh:** Game sử dụng đồ họa phong cách 2.5D. Dạng game top-down tương tự như *Soul Knight*, dạng đồ họa pixel, game có cơ chế Camera di chuyển theo Player qua đó có trải nghiệm tốt nhất. Trò chơi xây dựng hệ thống đa bản đồ với cơ chế chuyển tiếp liền mạch, cho phép người chơi di chuyển giữa các khu vực lãnh địa khác nhau sau khi hoàn thành nhiệm vụ.
- **Cấu trúc tạo nên Game:** Game sử dụng thư viện pygame của *Python* kết hợp với nhiều thư viện khác cùng các giải thuật và công thức toán học phức tạp.

1.2 Các CTDL và giải thuật đã được sử dụng

1.2.1 Nhóm giải thuật tìm đường và Trí tuệ nhân tạo (AI)

- **Breadth-First Search (BFS):**
 - *Định nghĩa:* Sử dụng hàng đợi (Queue) để loang từ vị trí thực thể trên lưới Grid để tìm đường đi ngắn nhất.
 - *Chức năng:* Đảm bảo quái vật (Enemies) có khả năng tự động tìm mục tiêu và tránh các vật cản (Walls, Obstacles) một cách chính xác trong môi trường Dungeon phức tạp.
- **Separation Algorithm (Thuật toán Tách biệt):**
 - *Định nghĩa:* Sử dụng Vector đẩy ngược chiều dựa trên khoảng cách vật lý giữa các thực thể.
 - *Chức năng:* Ngăn chặn tình trạng các thực thể (như Thức thần) đứng chồng lấp lên nhau, tạo hiệu ứng di chuyển bầy đàn tự nhiên.

1.2.2 Nhóm giải thuật Đồ họa và Hiển thị

- **Thuật toán sắp xếp**

- *Định nghĩa*: Là phương pháp sử dụng thuật toán **Sắp xếp (Sorting)** lên danh sách các Sprite dựa trên tọa độ trục Y (*centery*) của chúng trước mỗi khung hình vẽ. Ngoài ra còn dùng để sắp xếp thời gian để xếp hạng trên Leaderboard theo cơ chế ưu tiên Round -> Thời gian.
- *Chức năng*: Mô phỏng chiều sâu trong không gian 2.5D. Vật thể có tọa độ Y lớn hơn (đứng thấp hơn) sẽ được vẽ sau để che khuất vật thể ở trên, tạo hiệu ứng thị giác chân thực. Xây dựng Leaderboard cho Game.

- **Flood Fill (Giải thuật Loang màu):**

- *Định nghĩa*: Giải thuật loang bắt đầu từ một nút gốc để tìm và xác định các khu vực lân cận có cùng thuộc tính (màu sắc) trong một mảng hai chiều.
- *Chức năng*: Tự động nhận diện và loại bỏ các màu nền phức tạp (tách nền) từ Sprite Sheets đầu vào, giúp chuẩn hóa tài nguyên đồ họa cho game mà không làm hỏng chi tiết nội tại của nhân vật.

1.2.3 Nhóm Cấu trúc dữ liệu (Data Structures)

- **Singly Linked List (Danh sách liên kết đơn):**

- *Định nghĩa*: Cấu trúc dữ liệu tuyến tính bao gồm các nút (Nodes), mỗi nút chứa dữ liệu và một liên kết trỏ đến nút kế tiếp.
- *Chức năng*: Triển khai lớp `GhostNode` để quản lý hiệu ứng bóng ma di chuyển. Việc chèn vào đầu và xóa ở cuối danh sách giúp tối ưu hóa bộ nhớ khi xử lý chuỗi hình ảnh lịch sử của nhân vật.

- **Directed Acyclic Graph (DAG):**

- *Định nghĩa*: Đồ thị có hướng không có chu trình, dùng để biểu diễn các mối quan hệ phụ thuộc giữa các nút.
- *Chức năng*: Quản lý hệ thống Cây kỹ năng (Skill Tree). Đảm bảo logic nâng cấp theo đúng trình tự, người chơi phải hoàn thành kỹ năng tiền đề (Prerequisites) mới có thể mở khóa các kỹ năng cấp cao hơn.

2 Quá trình thực hiện

2.1 Tuần 1

2.1.1 Giai đoạn 1: Thiết lập nền tảng và Tiền xử lý đồ họa

- **Nghiên cứu và Khởi tạo:**

- Tìm hiểu thư viện Pygame, tập trung vào cơ chế *Game Loop* để đảm bảo sự đồng bộ giữa logic và hiển thị.
- Thiết lập hệ thống kiểm soát phiên bản Git, giúp quản lý mã nguồn và theo dõi các thay đổi trong quá trình phát triển.
- Xây dựng kịch bản, danh sách tính năng (To-do list) và phân tích các bài toán thuật toán trọng tâm.

- **Xây dựng và Tối ưu tài nguyên đồ họa:**

- Thu thập *Sprite Sheets* từ các nguồn mở. Để chuyển đổi các tấm ảnh thô thành hoạt ảnh, nhóm đã áp dụng kiến thức về cấu trúc dữ liệu và giải thuật.
- **Giải thuật Flood Fill (Stack-based DFS):** Triển khai thuật toán loang để tự động nhận diện vùng màu nền bao quanh nhân vật. Thuật toán này giúp tách biệt phần nhân vật chính, cắt nhỏ tấm ảnh thành các khung hình đơn lẻ có nền trong suốt, phục vụ cho việc tạo chuyển động (*Animation*) mượt mà.

2.1.2 Giai đoạn 2: Xây dựng cơ chế lõi và Quản lý không gian

- **Thiết kế hệ thống Bản đồ (Map System):**

- Xây dựng ma trận bản đồ (*Grid Matrix*) dựa trên cấu trúc mảng 2 chiều. Điều này giúp tối ưu hóa việc truy xuất dữ liệu ô (*Tile*) và quản lý các vùng va chạm tĩnh.

- **Quản lý hiển thị và Chiều sâu không gian:**

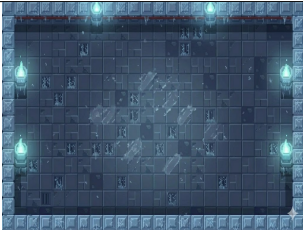
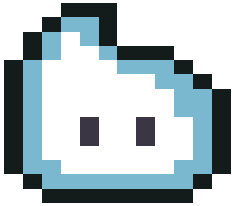
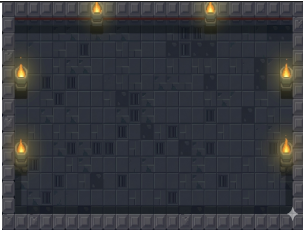
- Phát triển cơ chế *Camera Tracking* tự động bám đuổi nhân vật chính.
- **Thuật toán Y-Sorting:** Để giả lập không gian 2.5D, nhóm triển khai thuật toán sắp xếp thứ tự vẽ dựa trên tọa độ trục Y của *Hitbox*. Các vật thể có tọa độ Y lớn hơn sẽ được vẽ sau (đề lên), tạo cảm giác chiều sâu tự nhiên cho góc nhìn từ trên xuống (*Top-down*).

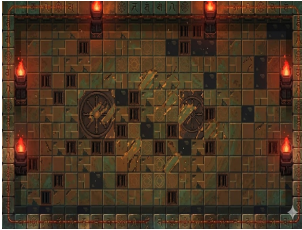
- **Cơ chế vận động và Hoạt ảnh:**

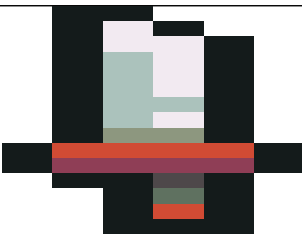
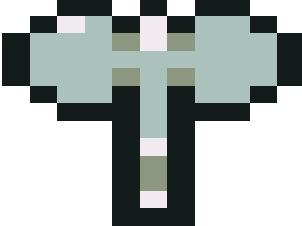
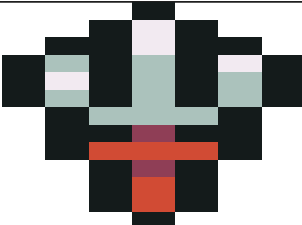
- Lập trình hệ thống di chuyển (*Movement*) sử dụng chuẩn hóa Vector để duy trì tốc độ không đổi theo mọi hướng.
- Xây dựng **Máy trạng thái hữu hạn (FSM)** để quản lý các bộ hoạt ảnh cho Player và Boss. Hệ thống cho phép chuyển đổi chính xác giữa các trạng thái như *Idle*, *Walk*, *Attack* và *Dash* dựa trên các điều kiện logic của nhân vật.
- Xử lý va chạm (*Collision*) thông qua việc tách biệt giữa khung hình hiển thị (*Rect*) và vùng va chạm thực tế (*Hitbox*), giúp các tương tác vật lý trở nên chính xác và chân thực hơn.

2.1.3 Danh sách hình ảnh nhân vật và map và vũ khí

Tên hình ảnh	Hình ảnh minh họa
Nhân vật chính	

Map round 1	
Boss round 1	
Quái round 1	
Quái round 1	
Map round 2	
Boss round 2	
Quái round 2	

Quái round 2	
Quái round 2	
Quái round 2	
Quái round 2	
Map round 3	
Boss round 3	
Quái round 3	

Quái round 3	
Quái round 3	
Vũ khí kiếm	
Vũ khí rìu	
Vũ khí sai	
Vũ khí thương	
Bình potion	

2.2 Tuần 2

2.2.1 Hệ thống chuyển động và Hiệu ứng hoạt ảnh

Trong giai đoạn này, nhóm tập trung vào việc thổi hồn cho các thực thể thông qua hệ thống Animation và các hiệu ứng vật lý đi kèm:

- **Chuyển động nhân vật và thực thể:** Thiết lập bộ hoạt ảnh di chuyển đa hướng cho Player, các loại quái vật và Boss. Sử dụng máy trạng thái hữu hạn (FSM) để đảm bảo việc chuyển đổi giữa các trạng thái (Idle, Walk, Dash) diễn ra mượt mà[cite: 147, 168].
- **Hiệu ứng đặc biệt (VFX):** Triển khai hệ thống bóng ma (*Afterimage*) thông qua cấu trúc dữ liệu *Singly Linked List* (GhostNode) khi Player thực hiện kỹ năng Dash hoặc khi Boss di chuyển tốc độ cao[cite: 68, 148].

2.2.2 Cơ chế chiến đấu và Hiệu ứng thuật thức

Hệ thống chiến đấu được nâng cấp với các hiệu ứng tương tác trực quan phản ánh trạng thái của đối tượng:

- **Hiệu ứng tấn công đa dạng:** Xây dựng các Sprite hiệu ứng cho hành động chém (*Slash*), cào (*Claw*) và va chạm vật lý dựa trên lớp `ParticleEffect`[cite: 92, 93].
- **Trạng thái thuật thức (Status Effects):**
 - **Đóng băng (Freeze):** Tạm dừng toàn bộ logic cập nhật của thực thể trong một khoảng thời gian nhất định[cite: 166, 192].
 - **Làm chậm (Slow):** Giảm tốc độ di chuyển của thực thể xuống 40% khi bị trúng đòn (ví dụ đòn tấn công từ Frog)[cite: 166, 194].
 - **Bị thương:** Hiệu ứng nhấp nháy (*Blink effect*) thông qua hàm `wave_value()` và hiệu ứng bật lùi (*Knockback*) dựa trên chỉ số *resistance* của từng loại quái[cite: 56, 179].

2.2.3 Cơ chế xây dựng bản đồ và Tương tác Hitbox

Hệ thống thế giới được xây dựng dựa trên sự phân tách giữa hiển thị và va chạm vật lý:

- **Dạng Map (Grid-based):** Sử dụng ma trận ký tự trong `settings.py` để mã hóa vị trí sàn, tường và các chướng ngại vật[cite: 27, 30].
- **Cơ chế Hitbox (Collision Logic):** Mọi thực thể đều có một hitbox riêng, thường được thu nhỏ (inflate) so với ảnh hiển thị (*Rect*) để tạo cảm giác chiều sâu 2.5D chính xác[cite: 38, 43].

Bảng các đối tượng tương tác trong hệ thống va chạm:

Đối tượng (Object)	Vai trò trong hệ thống Hitbox
Player	Thực thể trung tâm, có hitbox phản ứng với tường và đòn tấn công của địch[cite: 157].
Obstacle Sprites	Các vật cản tĩnh (cây, đá, đồ vật) ngăn chặn di chuyển của thực thể[cite: 135].
Wall (Tile)	Các ô biên hoặc tường ẩn định hình giới hạn di chuyển của bản đồ[cite: 38].
Attack Sprites	Hitbox tạm thời sinh ra từ vũ khí hoặc phép thuật để kiểm tra va chạm gây sát thương[cite: 58, 136].
Attackable Sprites	Nhóm các thực thể có thể nhận sát thương (Enemies, Summons)[cite: 136].

2.3 Tuần 3: Hệ thống chỉ số, AI nâng cao và Hoàn thiện vòng lặp Gameplay Round 1 2

2.3.1 Hệ thống dữ liệu và Hiển thị nội suy (Juice & Feedback)

Thiết lập nền tảng chỉ số thực thể và tối ưu hóa phản hồi thị giác:

- **Cấu hình chỉ số (Status System):** Thiết lập hệ thống dữ liệu tại `settings.py` cho Player và Monster (HP, Armor, Attack, Cooldown, Speed, Resistance).
- **Nội suy thanh trạng thái (Lerp Logic):** Xây dựng thanh Máu và Giáp với hiệu ứng giảm mượt mà thông qua hàm nội suy tuyến tính.
- **Phản hồi chiến đấu:** Tích hợp thanh "ghost bar" và các hiệu ứng rung màn hình (*Screen Shake*), nhấp nháy (*Wave blink*) khi thực thể nhận sát thương.

2.3.2 Hoàn thiện Round 1: Hệ thống AI và Cơ chế Boss Băng

Tập trung vào việc xây dựng trí tuệ nhân tạo cho nhóm quái vật tiên phong và cơ chế chiến đấu đặc thù của Boss:

- **Thực thể đối địch (Enemies):**
 - **Spirit & Slime:** Hai loại quái vật cơ bản sử dụng thuật toán BFS để bám đuổi người chơi. Slime có khả năng phân tách hoặc gây chậm, trong khi Spirit có tốc độ di chuyển cao.
 - **Giant Ice Boss (Boss Băng):** Thực thể có kích thước siêu trọng, sở hữu cơ chế **Freeze** (Đóng băng). Khi Player nằm trong vùng ảnh hưởng của đòn tấn công, Boss sẽ kích hoạt trạng thái đóng băng, vô hiệu hóa hoàn toàn khả năng di chuyển và tấn công của người chơi trong một khoảng thời gian ngắn.
- **Cơ chế nhận diện và Tương tác (Radius Logic):** Nhóm triển khai hệ thống quản lý khoảng cách để điều khiển hành vi quái vật thông qua hai thông số cốt lõi:
 - **Notice Radius (Bán kính chú ý):** Ngưỡng khoảng cách mà tại đó AI bắt đầu chuyển từ trạng thái *Idle* sang *Chase*. Khi người chơi lọt vào vùng này, quái vật sẽ kích hoạt thuật toán BFS để tìm đường bám đuổi.
 - **Attack Radius (Bán kính tấn công):** Ngưỡng khoảng cách tối thiểu để quái vật dừng di chuyển và bắt đầu thực hiện hoạt ảnh tấn công (*Attack animation*). Mỗi thực thể (Spirit, Slime, Boss) có một bán kính tấn công riêng biệt để phù hợp với tầm đánh của chúng.

- **Thuật toán Tìm đường và Máy trạng thái (FSM):** Triển khai hàm `get_bfs_direction` trên lưới 24×16 ô, giúp quái vật tự động né tránh vật cản. FSM được thiết lập để quản lý sự chuyển đổi linh hoạt giữa các trạng thái: *Idle* (Nghỉ), *Chase* (Truy đuổi bằng BFS) và *Attack* (Tấn công dựa trên Radius).

2.3.3 Hoàn thiện Round 2: Boss Megumi và Thuật thức Triệu hồi

Nâng cấp logic chiến đấu với hệ thống triệu hồi phức tạp:

- **AI Boss Megumi (Summoner):**
 - **Mana Regeneration:** Cơ chế tự hồi năng lượng để thi triển kỹ năng.
 - **Chiến thuật Triệu hồi:** Boss tự tính toán số lượng quái hiện có để triệu hồi thêm các loại Thức thần (Chó đen, Chó trắng,Ếch hoặc Bò).
 - **Cơ chế Hợp thể (Fusion):** Khi cặp Divine Dogs cùng hiện diện, Boss triệu hồi **Totality Dog** với sức mạnh vượt trội.
- **Hệ thống Thức thần (Player's Summons):**
 - Triệu hồi Divine Dogs thông qua vũ khí *Sai* với chỉ số tỉ lệ thuận (*Scaling*) theo chỉ số người chơi.
 - **Obstacle Logic:** Thức thần đóng vai trò vật cản vật lý chặn đường kẻ thù và tự động hồi phục HP khi ở trạng thái chờ.
 - **Xử lý tử trận:** Khi Boss bị hạ, toàn bộ thức thần của Boss tự động giải trừ (*Despawn*).

2.3.4 Cơ chế Combat và Hiệu ứng Trạng thái chuyên sâu

- **Nhắm mục tiêu Động (Dynamic Targeting):** Triển khai hệ thống *Aggro*, quái vật tự chọn mục tiêu gần nhất giữa Player và Thức thần đồng minh.
- **Hiệu ứng trạng thái (Status Effects):** Áp dụng các trạng thái *Burn* (Bỏng), *Slow* (Làm chậm) và *Freeze* (Đóng băng từ vũ khí Axe).
- **Tính toán sát thương:** Áp dụng công thức giảm trừ qua giáp: $Damage_{final} = Damage_{base} - Armor$.

2.3.5 Quản lý Màn chơi và Tiền xử lý đồ họa tự động

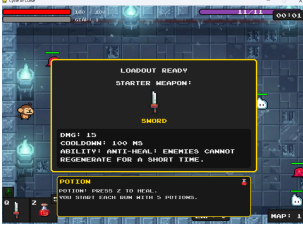
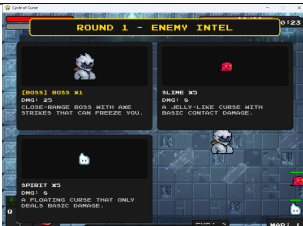
Hoàn thiện hệ thống quản lý luồng game và tối ưu hóa tài nguyên nạp vào:

- **Logic Mở cổng (Gate Logic):** Xây dựng bộ đếm thực thể tự động. Hệ thống chỉ tính toán số lượng quái vật thù địch (loại trừ Thức thần của người chơi) để kích hoạt cổng dịch chuyển sang màn tiếp theo khi toàn bộ kẻ thù bị tiêu diệt.
- **Hệ thống Giới thiệu Round (Intro Sequence):** Phát triển giao diện chuyển cảnh, tự động hiển thị thông tin chi tiết về các loại quái vật và vũ khí mới mỗi khi bắt đầu một Round chơi mới.
- **Xử lý Đồ họa tự động (Flood Fill):** Để tối ưu hóa quy trình thêm tài nguyên, nhóm triển khai thuật toán **Flood Fill** (loang màu) để tự động nhận diện và tách nền (*Background Removal*) cho các Sprite quái vật và vũ khí ngay khi nạp vào bộ nhớ, đảm bảo hình ảnh hiển thị trong suốt và đồng bộ với môi trường game.

Bảng chi tiết các chỉ số cấu hình trong mã nguồn:

Nhóm chỉ số	Biến số	Ý nghĩa kỹ thuật
Tấn công	damage	Lượng máu bị trừ khi va chạm hitbox.
Phòng thủ	armor	Giá trị giảm trừ sát thương trực tiếp.
Tầm nhìn AI	radius	Khoảng cách tối đa để quái vật kích hoạt BFS.
Kháng đòn	resistance	Khả năng chống chịu lực đẩy ngược (<i>Knockback</i>).
Tự động nạp	regen	Tốc độ hồi phục Mana (Boss) hoặc HP (Thực thể).

2.3.6 Danh sách hình ảnh minh họa

Tên hình ảnh	Hình ảnh minh họa
Ảnh giới thiệu vũ khí map 1	
Gameplay map 1	
Giới thiệu quái map 1	
Giới thiệu quái map 2	

Gameplay map 2



2.4 Tuần 4: Hoàn thiện Boss Sukuna, Hệ thống Skill Tree và Menu khởi đầu

2.4.1 Hệ thống Boss Sukuna và Round cuối (Final Round)

Phát triển thực thể mạnh nhất trong trò chơi với các cơ chế chiến đấu phức tạp và hiệu ứng diện rộng:

- **Cơ chế Boss Sukuna:** Xây dựng hệ thống chỉ số vượt trội với lượng HP và Armor cực lớn. Boss được thiết lập máy trạng thái (FSM) nâng cao, có khả năng tung ra các đòn đánh diện rộng (*AOE - Area of Effect*) gây sát thương trên phạm vi lớn, buộc người chơi phải sử dụng kỹ năng *Dash* liên tục để né tránh.
- **Cơ chế Phóng hỏa (Fire Manipulation):** Triển khai hệ thống đạn đạo (*Projectile system*) cho các đợt tấn công từ xa. Lửa được bắn ra theo hướng người chơi, đi kèm với hiệu ứng hạt (*Particles*) và trạng thái *Burn* (Bỏng) gây sát thương theo thời gian.

2.4.2 Hệ thống Menu và Cây kỹ năng (Skill Tree)

Xây dựng giao diện người dùng (UI) và cơ chế nâng cấp chiều sâu cho Gameplay:

- **Hệ thống Menu chính:** Thiết lập màn hình bắt đầu với các lựa chọn: *Start Game*, *Leaderboard* và *Exit*. Sử dụng các hiệu ứng chuyển cảnh mượt mà khi bắt đầu vào các Round chơi.
- **Cây kỹ năng (Skill Tree):**
 - Xây dựng giao diện nâng cấp cho phép người chơi sử dụng điểm thưởng sau mỗi màn để gia tăng các chỉ số: *Damage*, *Max Health*, *Speed*, *Armor* và giảm *Cooldown*.
 - Hệ thống tự động cập nhật lại biến dữ liệu trong `player_stats` ngay khi người chơi thực hiện nâng cấp thành công.

2.4.3 Hệ thống Lưu trữ kỷ lục (Leaderboard System)

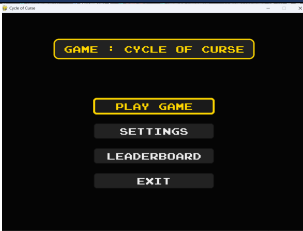
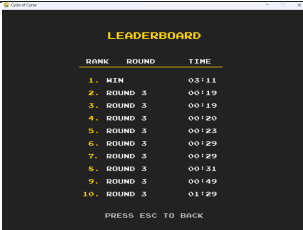
Tích hợp khả năng quản lý dữ liệu bền vững cho trò chơi:

- **Cơ chế ghi nhận thời gian:** Hệ thống tự động bắt đầu bộ đếm khi Round 1 khởi động và dừng lại ngay khi Boss Sukuna bị tiêu diệt.
- **Lưu trữ dữ liệu JSON:** Triển khai module `json` để tự động ghi lại kỷ lục thời gian hoàn thành màn chơi (*Clear Time*) của người chơi vào file `leaderboard.json`.
- **Hiển thị bảng xếp hạng:** Xây dựng logic đọc file JSON và sắp xếp dữ liệu (*Sort*) để hiển thị danh sách các kỷ lục tốt nhất lên màn hình Leaderboard tại Menu chính.

2.4.4 Tổng kết và Đóng gói sản phẩm

- Kiểm tra toàn bộ hệ thống va chạm (Hitbox) giữa các thực thể cuối game để đảm bảo không xảy ra lỗi logic.
- Tối ưu hóa hiệu suất vẽ (*Rendering*) bằng cách loại bỏ các đối tượng nằm ngoài khung hình Camera.
- Đóng gói mã nguồn và chuẩn bị báo cáo kỹ thuật chi tiết về các thuật toán DSA (BFS, Flood Fill) đã áp dụng.

2.4.5 Danh sách hình ảnh minh họa

Tên hình ảnh	Hình ảnh minh họa
Menu khởi đầu	
Bảng xếp hạng	
Setting	
Giới thiệu quái map 3	
Cây kỹ năng	

3 Kết quả đạt được

Link đồ án trên GitHub: https://github.com/sherwyndd/DSA_GAME_CycleOfCurse
Sau quá trình nghiên cứu và phát triển dự án game *Cycle of Curse*, tôi đã hoàn thành các mục tiêu đề ra với những kết quả cụ thể như sau:

3.1 Về mặt Kỹ thuật và Giải thuật (Technical & Algorithms)

- **Xử lý không gian và Vật lý:** Hoàn thiện hệ thống giả lập không gian 2.5D bằng thuật toán **Y-Sorting**. Xử lý va chạm chính xác thông qua việc tách biệt *Hitbox* (vùng va chạm thực) và *Rect* (khung hình hiển thị).
- **Trí tuệ nhân tạo (AI):**
 - Triển khai thành công giải thuật **BFS (Breadth-First Search)** giúp quái vật tìm đường thông minh, vượt vật cản trên bản đồ lưới.
 - Quản lý hành vi thực thể chặt chẽ thông qua **Máy trạng thái hữu hạn (FSM)** với các trạng thái phức tạp như *Chase*, *Attack*, *Hurt*, *Despawn*.
- **Tối ưu hóa tài nguyên:** Ứng dụng giải thuật **Flood Fill** để tự động hóa việc tiền xử lý đồ họa (tách nền Sprite), giúp giảm thiểu thời gian xử lý khi thêm tài nguyên mới.

3.2 Về mặt Gameplay và Tính năng (Gameplay Features)

- **Hệ thống màn chơi (Level Design):** Xây dựng hoàn chỉnh 3 vòng đấu (*Round*) với độ khó tăng dần, đi kèm hệ thống **Gate Logic** tự động quản lý luồng màn chơi.
- **Cơ chế chiến đấu độc đáo:**
 - Phát triển hệ thống Boss đa dạng: *Giant Ice Boss* (đóng băng), *Megumi* (triệu hồi/hợp thể), và *Sukuna* (tấn công diện rộng AOE).
 - Triển khai cơ chế **Thức thần (Summoning System)** cho người chơi với các thuật toán *Stat Scaling* (tỉ lệ chỉ số) và *Obstacle Logic* (vật cản chiến thuật).
- **Hệ thống tiến triển:** Hoàn thiện giao diện **Skill Tree** cho phép nâng cấp chỉ số nhân vật bền vững sau mỗi trận đấu.

3.3 Về mặt Trải nghiệm và Dữ liệu (UX & Data)

- **Phản hồi thị giác (Game Juice):** Đạt độ mượt mà cao trong hiển thị nhờ cơ chế **Nội suy (Lerp)** thanh trạng thái, kết hợp các hiệu ứng rung màn hình, nhấp nháy đỏ và hệ thống hạt (*Particles*).
- **Quản lý dữ liệu:** Tích hợp hệ thống **Leaderboard** sử dụng tệp tin JSON, cho phép lưu trữ và truy xuất kỷ lục thời gian hoàn thành màn chơi của người dùng một cách ổn định.

4 Tài liệu tham khảo

1. Video hướng dẫn lập trình RPG với Pygame:
<https://www.youtube.com/watch?v=QU1pPzEGrqw&t=27166s>
2. Cộng đồng thảo luận ý tưởng Soul Knight x Jujutsu Kaisen (Phần 1):
https://www.reddit.com/r/SoulKnight/comments/1ak8ih1/soul_knight_x_jjk/#lightbox
3. Cộng đồng thảo luận ý tưởng Soul Knight x Jujutsu Kaisen (Phần 2):
https://www.reddit.com/r/SoulKnight/comments/1ic349m/soul_knight_x_jjk_part2/
4. Kho tài nguyên Pixel Art và Hiệu ứng hạt (Particle assets):
<https://itch.io/game-assets/tag-particle/tag-pixel-art>
5. Công cụ thiết kế đồ họa Pixel trực tuyến (Pixilart):
<https://www.pixilart.com/draw>

A Phụ lục 1: Giới thiệu (demo) kết quả

Đường dẫn xem video demo chi tiết:

<https://drive.google.com/file/d/1W4G8rhbFBVlQTJe0drnjx0o6QgbG5gC/view?usp=sharing>

B Phụ lục 2: Docstring

Phụ lục này trình bày toàn bộ docstring cho 14 module của dự án *Cycle of Curse*, bao gồm mô tả cấp module, class, phương thức và các thuật toán/cấu trúc dữ liệu (DSA) được triển khai.

Kiến trúc tổng thể: `main.py` (Game loop) → `level.py` (World orchestration) → `player.py`, `enemy.py`, `summon.py`, `weapon.py`, `magic.py`, `particles.py`, `ui.py`; với `settings.py`, `support.py`, `tile.py`, `entity.py` là các module hỗ trợ.

B.1 `main.py` — Entry Point & Game Loop

Mô tả module: Điểm khởi đầu của toàn bộ ứng dụng game. Quản lý Application State Machine với 4 trạng thái: `MENU` → `SETTINGS` / `LEADERBOARD` / `GAME` → `MENU`. Khởi tạo Pygame, tạo cửa sổ, duy trì vòng lặp chính ở FPS cố định.

DSA: FSM (Finite State Machine) — 4 trạng thái ứng dụng với luồng chuyển đổi rõ ràng.

Class Game

Chức năng: Controller vòng lặp game chính. Khởi tạo và điều phối Menu, Leaderboard, Settings, Level.

Attributes:

- `state` — Trạng thái hiện tại: `'MENU'`, `'GAME'`, `'SETTINGS'`, `'LEADERBOARD'`.
- `level` — Instance Level đang chạy; None khi ở menu.

- `clock` — `pygame.time.Clock`, giới hạn FPS.

Phương thức:

- `__init__()` — Khởi tạo Pygame, cửa sổ hiển thị và tắt cả sub-system.
- `check_controls_ready()` — Kiểm tra tất cả phím trong `CONTROLS` đã được gán (không có `None`), trả về `bool`.
- `run()` — Vòng lặp game chính: xử lý events → update state → draw → `clock.tick(FPS)`.

B.2 settings.py — Cấu hình Trung tâm

Mô tả module: File cấu hình trung tâm — mọi hằng số và dữ liệu tĩnh đều định nghĩa tại đây. Không chứa class hay logic, chỉ chứa biến và dict để các module khác import.

Các hằng số và cấu trúc dữ liệu chính:

- `WIDTH, HEIGHT, FPS` — Kích thước cửa sổ (pixel) và tốc độ khung hình.
- `COLS, ROWS, T_WIDTH, T_HEIGHT` — Kích thước lưới bản đồ: 24×16 ô, mỗi ô 51×44 px.
- `weapon_data` — dict thống kê vũ khí: cooldown, damage, đường dẫn graphic.
- `magic_data` — dict thống kê phép thuật: strength, cost, đường dẫn graphic.
- `monster_data` — dict thống kê đầy đủ từng loại quái: HP, damage, speed, radius, EXP.
- `CONTROLS` — dict mapping phím điều khiển, có thể ghi đè qua màn hình Settings.
- `MAPS` — dict cấu hình từng màn: layout matrix, background, vị trí gate.

B.3 support.py — Hàm Tiện ích

Mô tả module: Các hàm tiện ích dùng chung trong toàn bộ codebase. Không có class, chỉ chứa 2 hàm độc lập.

DSA: Flood Fill (Stack-based DFS) — `remove_background_floodfill` duyệt pixel bằng stack, độ phức tạp $O(W \times H)$.

Hàm:

- `import_folder(path)` — Nạp tất cả ảnh trong thư mục `path`, trả về `list[Surface]` sắp xếp theo tên file. Dùng cho animation frames.
- `remove_background_floodfill(surf, threshold=15)` — Xóa nền ảnh bằng Flood Fill từ 4 góc, ngưỡng màu `RGB = threshold`. Trả về `Surface` đã sửa in-place.

B.4 tile.py — Ô Bản đồ Tĩnh

Mô tả module: Định nghĩa lớp `Tile` — đơn vị ô cơ bản xây dựng bản đồ game. Hitbox thu nhỏ 10px theo chiều dọc để tạo cảm giác chiều sâu 2.5D. `sprite_type` quyết định hành vi: `'object'` = vật cản, `'invisible'` = biên ẩn.

`Class Tile(pygame.sprite.Sprite)`

Chức năng: Sprite tĩnh đại diện cho một ô bản đồ. Không có logic update.

Attributes:

- `sprite_type` — 'object', 'invisible', 'grass', 'floor'.
- `hitbox` — Rect va chạm (inflate -10px theo Y).

B.5 entity.py — Lớp Cơ sở Entity

Mô tả module: Lớp cơ sở trừu tượng cho mọi thực thể di chuyển: `Player`, `Enemy`, `Summon`. Cung cấp hệ thống di chuyển, va chạm và hiệu ứng nhấp nháy khi bị đánh.

DSA:

- **Overlap Comparison** — so sánh overlap 2 phía để quyết định hướng đẩy ngược, $O(n)$ theo số obstacle sprites, tránh thực thể bị “dính tường”.
- **Vector Normalization** — chuẩn hóa `direction` để tốc độ đường chéo bằng tốc độ thẳng.

`Class Entity(pygame.sprite.Sprite)`

Attributes:

- `frame_index` — Chỉ số khung animation hiện tại (float, tăng theo `animation_speed`).
- `animation_speed` — Tốc độ chạy animation (khung/tick).
- `direction` — `pygame.math.Vector2` hướng di chuyển hiện tại.

Phương thức:

- `move(speed)` — Di chuyển theo `direction`, chuẩn hóa vector, kiểm tra collision 2 trục, clamp biên bản đồ.
- `collision(direction)` — Xử lý va chạm theo 'horizontal'/'vertical', dùng min-overlap rule để quyết định hướng đẩy ngược.
- `wave_value()` — Trả về 255/0 theo sin thời gian, tạo blink effect khi bị đánh.

B.6 weapon.py — Sprite Vũ khí

Mô tả module: Định nghĩa lớp `Weapon` — sprite vũ khí xuất hiện khi `Player` hoặc `Boss` tấn công. Tự tạo và tự hủy theo animation attack của owner. Loại vũ khí: *sword*, *lance*, *axe*, *rapier*, *sai*.

`Class Weapon(pygame.sprite.Sprite)`

Chức năng: Sprite vũ khí cận chiến bám theo owner. Nền ảnh được xóa bằng Flood Fill trước khi hiển thị.

Phương thức:

- `__init__(owner, groups)` — Nạp ảnh, xóa nền, đặt vị trí ban đầu theo hướng owner.
- `update_position()` — Cập nhật tọa độ mỗi frame: neo vào cạnh tương ứng của rect owner (midright/left/bottom/top).

B.7 magic.py — Hệ thống Phép thuật

Mô tả module: Hệ thống phép thuật của **Player**: *heal* (hồi máu) và *dismantle* (phóng SukunaSlash). SukunaSlash di chuyển thẳng, có hiệu ứng shimmer (scale sin) và trail bóng ma.

DSA: Singly Linked List (`GhostNode.next`) — quản lý chuỗi afterimage của SukunaSlash. Mỗi frame thêm node đầu, node đuôi tự xóa khi $\alpha \leq 0$, độ phức tạp $O(k)$ với k là số bóng ma tồn tại đồng thời.

Class GhostNode

Attributes:

- `surf` — Bản sao Surface của slash tại thời điểm tạo node.
- `rect` — Vị trí tại thời điểm tạo.
- `alpha` — Độ trong suốt (giảm dần mỗi frame).
- `next` — Con trỏ node tiếp theo trong Linked List.

Class SukunaSlash(Entity)

Chức năng: Projectile di chuyển thẳng, kéo trail bóng ma qua Linked List.

Phương thức:

- `update_ghosts()` — Thêm `GhostNode` mới vào đầu; giảm `alpha` từng node; xóa node đuôi khi $\alpha \leq 0$.
- `draw_ghosts(surface, offset)` — Vẽ toàn bộ chuỗi bóng ma lên surface với camera offset.

Class MagicPlayer

Chức năng: Controller điều phối cast phép thuật.

Phương thức:

- `heal(player, strength, cost, groups)` — Cộng vào `target_health` (hệ thống nội suy smooth HP bar).
- `dismantle(player, groups, obstacle_sprites)` — Phóng SukunaSlash theo hướng player đang đứng.

B.8 particles.py — Hiệu ứng Hạt

Mô tả module: Quản lý particle effects và one-shot animations qua `AnimationPlayer` và `ParticleEffect`. Nạp sẵn toàn bộ frame từ đĩa khi khởi động, hỗ trợ tint màu và scale.

Class `AnimationPlayer`

Attributes:

- `frames` — dict {tên: list[Surface]} nạp sẵn lúc khởi tạo.

Phương thức:

- `create_particles(type, pos, groups, pos_type)` — Spawn `ParticleEffect` tại `pos`. `pos_type`: 'center' hoặc 'midbottom'.
- `reflect_images(frames)` — Flip frames horizontally cho boss sprites bị ngược chiều.

Class `ParticleEffect(pygame.sprite.Sprite)`

Chức năng: Sprite animation một lần: chạy hết frame thì tự `kill()`. Có `hitbox = rect` để `YSortCameraGroup` depth-sort đúng.

B.9 menu.py — Màn hình Menu

Mô tả module: Các màn hình ngoài gameplay: *Main Menu*, *Settings* và *Leaderboard*. Điều hướng bằng bàn phím W/S hoặc mũi tên.

DSA:

- FSM nhỏ trong *Settings*: idle → waiting_for_key → captured → idle.
- Sort ascending (time) trong *Leaderboard* để hiển thị top 5 nhanh nhất.

Class `Menu`

Chức năng: Màn hình chủ với 4 nút, nút được chọn phát hiệu ứng pulse sin vàng.

Class `Settings`

Chức năng: Rebind phím điều khiển, đổi nhân vật và vũ khí mặc định. Lưu vào `CONTROLS` (dict toàn cục trong `settings.py`).

Phương thức:

- `handle_input(event)` — Xử lý event keyboard để rebind hoặc đổi nhân vật/vũ khí. Quản lý FSM `waiting_for_key`.

Class `Leaderboard`

Chức năng: Lưu và hiển thị top 5 lần chơi nhanh nhất từ file `leaderboard.txt`.

Phương thức:

- `save_record(round, time)` — Thêm record mới, sort ascending theo time, giữ top 5, ghi file.

B.10 ui.py — Giao diện HUD

Mô tả module: Toàn bộ HUD và màn hình phụ trong gameplay. Tất cả method `display/show_*` vẽ trực tiếp lên `display_surface` và được gọi mỗi frame từ `Level.run()`.

DSA:

- **DAG Prerequisite (Skill Tree):** node chỉ unlock khi mọi prereq đã mua, kiểm tra dependency graph trước khi cho phép.
- **Smooth HP interpolation:** `target_amount` nội suy về `current` qua nhiều frame, tạo thanh HP mượt mà.

Class UI

Phương thức:

- `show_bar(current, max, rect, color, target)` — Vẽ thanh HP/Armor với smooth interpolation bằng thanh con xám.
- `draw_skill_tree(player, cat_idx, skill_idx, mode)` — Hiển thị và xử lý tương tác cây kỹ năng; kiểm tra DAG prerequisite trước khi unlock.
- `show_reward(player)` — Màn hình thưởng sau khi clear màn.
- `show_round_enemy_intro(enemies)` — Intro giới thiệu quái trước khi bắt đầu vòng mới.
- `show_game_over()` — Màn hình Game Over với nút Play Again / Back to Menu.
- `show_win()` — Màn hình thắng với thống kê thời gian và ghi leaderboard.

B.11 level.py — Điều phối Game World

Mô tả module: Lớp trung tâm điều phối toàn bộ thế giới game. Quản lý sprite groups, combat, chuyển màn và là cầu nối Player ↔ Enemy ↔ UI ↔ AnimationPlayer.

DSA:

- **Y-Sort (YSortCameraGroup):** sắp xếp visible sprites theo `hitbox.centery` mỗi frame, giả lập chiều sâu 2.5D.
- `spritecollide()`: $O(n \cdot m)$ kiểm tra va chạm giữa attack và attackable sprites.
- **Map Transitions:** FSM chuyển trạng thái màn với spawn position handling.

Class Level

Attributes:

- `status` — 'playing', 'game_over', 'win', 'back_to_menu'.
- `current_map` — 'first', 'second', 'fourth'.
- `visible_sprites` — YSortCameraGroup, vẽ theo Y depth.
- `obstacle_sprites` — Group sprite chướng ngại vật cho collision.
- `attack_sprites / attackable_sprites` — Groups cho combat logic.

Phương thức:

- `create_map()` — Đọc `WORLD_MAP` matrix, spawn `Tile/Enemy/Player` theo ký hiệu ô.
- `switch_map(new_map, spawn_pos)` — Xóa sprites cũ, tải bản đồ mới, đặt lại spawn player.
- `player_attack_logic()` — Kiểm tra va chạm `attack` → `attackable`; áp damage + status effects.
- `damage_player(amount, attack_type)` — Áp damage lên Player sau armor, kích hoạt particles + invincibility frame.
- `run(events)` — Vòng lặp Level: `input` → `update` → `combat` → `draw` → `UI`.
- `consume_leaderboard_save()` — Trả về dict leaderboard entry cho `Game.run()` lưu.

Class `YSortCameraGroup(pygame.sprite.Group)`

Chức năng: Custom sprite group sắp xếp sprites theo Y để giả lập depth 2.5D. Camera bù offset theo vị trí player.

Phương thức:

- `custom_draw(player)` — Sort sprites theo `hitbox.centery`, vẽ theo thứ tự đó.
- `enemy_update(player)` — Gọi `enemy_update()` cho tất cả enemy sprites.

B.12 `player.py` — Nhân vật Người chơi

Mô tả module: Định nghĩa lớp `Player`. 3 nhân vật lựa chọn: *Monkey*, *Megumi*, *Sukuna*. FSM trạng thái: *idle* / *walk* / *attack* / *dash*. Skill Tree 20+ node với DAG prerequisite, status effects, smooth HP bar.

DSA:

- **Singly Linked List (GhostNode):** afterimage khi dash, $O(k)$ update/frame.
- **FSM (get_status):** ưu tiên `attack` > `dash` > `walk` > `idle`.
- **DAG Prerequisite (can_unlock_skill):** kiểm tra toàn bộ prereq trước khi cho mua skill.

Class `GhostNode`

Chức năng: Node đơn trong Linked List afterimage của Player khi dash.

Attributes:

- `surf` — Bản sao Surface player tại thời điểm tạo.
- `rect` — Vị trí player lúc tạo node.
- `alpha` — Độ mờ (giảm dần).
- `next` — Con trỏ node tiếp theo.

Class Player(Entity)

Attributes:

- `health / target_health` — HP hiện tại và HP đích để nội suy smooth bar.
- `armor` — Giáp, giảm damage nhận vào.
- `exp` — Điểm kinh nghiệm để mua skill.
- `status` — FSM state dạng '`{direction}_{state}`', ví dụ '`right_attack`'.
- `skill_tree_unlocked` — dict `{node_id: bool}` trạng thái unlock từng skill.

Phương thức:

- `get_movement_input()` — Đọc CONTROLS và cập nhật `direction`. Khóa khi frozen/attacking.
- `get_attack_input()` — Xử lý ATTACK/MAGIC/SWITCH key, tạo Weapon hoặc cast magic.
- `get_status()` — Cập nhật FSM: `attack > dash > walk > idle`.
- `can_unlock_skill(node)` — Kiểm tra DAG prereq + đủ EXP + chưa mua, trả về bool.
- `unlock_skill(node_id)` — Trừ EXP, đánh dấu unlock, gọi `apply_skill_effect()`, trả về bool.
- `move(speed)` — Override `Entity.move()`: thêm dash ($\times 3.5$) và slow (40%) trước khi gọi super.
- `cooldowns()` — Quản lý tất cả cooldown timers và hết hạn status effects mỗi frame.
- `animate()` — Tăng `frame_index`; kết thúc attacking khi hết animation; áp wave blink.
- `update_ghosts()` — Linked List: thêm node khi dash, giảm alpha, xóa khi $\alpha \leq 0$.
- `freeze()` — Áp Freeze: dừng chuyển động, hủy tấn công. Tự hết sau `freeze_duration`.
- `update()` — Gọi toàn bộ hệ thống mỗi frame: `input` $\rightarrow$ `status` $\rightarrow$ `animate` $\rightarrow$ `cooldowns` $\rightarrow$ `ghosts`.

B.13 enemy.py — AI Kẻ thù

Mô tả module: Định nghĩa lớp `Enemy` — AI kẻ thù với BFS pathfinding và FSM hành vi. Hỗ trợ nhiều loại quái: *spirit*, *slime*, *skeleton*, *shaman*, *bull*, *frog* và 3 boss. FSM: `idle` $\rightarrow$ `notice` $\rightarrow$ `move` $\rightarrow$ `attack` $\rightarrow$ `cooldown` $\rightarrow$ `move`...

DSA:

- **BFS Pathfinding** (`get_bfs_direction`): xây dựng lưới ô 24×16 từ `obstacle_sprites`, chạy BFS từ vị trí quái đến mục tiêu, $O(384)$ mỗi lần gọi.
- **Dynamic Target Priority**: ưu tiên mục tiêu gần nhất trong `[player, summons]`.
- **Spatial Grid**: chuyển tọa độ pixel $\rightarrow$ ô lưới (`col, row`) cho BFS.

Class Enemy(Entity)

Attributes:

- `monster_name` — Khóa tra cứu trong `monster_data`.
- `health / max_health` — HP hiện tại và tối đa.
- `status` — FSM state: 'idle', 'move', 'attack'.
- `vulnerable` — False khi đang trong invincibility frame sau khi bị đánh.
- `resistance` — Hệ số bật lùi khi bị đánh (cao = ít bị đẩy).

Phương thức:

- `get_target_distance_direction(targets)` — Tìm target gần nhất, trả về (`distance`, `direction`, `target`).
- `get_status(targets)` — FSM: idle/move/attack theo distance so với notice/attack radius.
- `actions(targets)` — Thực hiện hành động: BFS move / tấn công / idle theo status.
- `get_bfs_direction(targets)` — BFS 4 hướng trên lưới, trả về `Vector2` hướng bước kế tiếp.
- `get_damage(player, attack_type)` — Nhận damage: bật invincible frame, cập nhật hướng bật lùi.
- `hit_reaction()` — Bật lùi: $\text{direction} \times (-\text{resistance})$.
- `check_death()` — $\text{HP} \leq 0$: despawn summons, trao EXP, particles, `kill()`.
- `enemy_update(targets)` — Gọi `get_status()` + `actions()` mỗi frame.

B.14 `summon.py` — Shikigami (Linh thú)

Mô tả module: Định nghĩa các shikigami triệu hồi: `DivineDog` (cơ sở), `Frog` và `Bull`. Hỗ trợ 2 chế độ: boss-owned (tấn công Player) và player-owned (tấn công Enemy). Vòng đời: Spawn animation → Orbit/Chase → Attack → Despawn animation.

DSA:

- **BFS Pathfinding 8 hướng** (`get_bfs_direction`): hỗ trợ di chuyển chéo, kiểm tra tránh cắt góc tường.
- **Separation Algorithm** (`_orbit_owner`): repulsion giữa các summon đang chồng lên nhau, $O(k^2)$ với k summon.
- **FSM Lifecycle:** spawning → orbiting → chasing → attacking → despawning.

Class DivineDog(Entity)

Chức năng: Shikigami cơ sở, kế thừa Entity cho move/collision. Có procedural spawn/despawn animation (scanline reveal/erase).

Attributes:

- `is_player_owned` — True nếu được Player triệu hồi.
- `orbit_angle` / `orbit_radius` — Góc và bán kính quỹ đạo quanh owner.
- `spawn_state` — FSM vòng đời: 'spawning' / 'alive' / 'despawning'.

Phương thức:

- `get_bfs_direction(target_pos)` — BFS 8 hướng, trả về `Vector2` hướng kế tiếp, tránh cắt góc tường.
- `_get_target_pos()` — Boss-owned: Player pos. Player-owned: enemy gần nhất.
- `_chase_target()` — BFS đến target, leash giới hạn khoảng cách với owner.
- `_orbit_owner(dt)` — Bay vòng theo đường tròn + separation repulsion với summon khác.
- `_try_attack()` — Kiểm tra tầm + cooldown rồi áp damage lên target.
- `begin_despawn()` — Chuyển sang state despawning, kích hoạt animation xóa dần.
- `enemy_update(player)` — Gọi mỗi frame từ `YSortCameraGroup`: update state + hành động.

Class Frog(DivineDog)

Chức năng: Nhảy cóc theo sóng sin ngang. Gây slow khi va chạm Player.

Phương thức:

- `_try_attack()` — Kiểm tra `spritecollide` trực tiếp thay vì tấn công theo tầm.
- `update()` — Override: áp sin wave lên tọa độ Y để tạo hiệu ứng nhảy.

Class Bull(DivineDog)

Chức năng: Lao thẳng theo hướng cố định khi triệu hồi. Có thể xuyên nhiều kẻ thù liên tiếp trong một lần lao.

Attributes:

- `charge_direction` — `Vector2` hướng cố định từ lúc triệu hồi, không thay đổi.

Phương thức:

- `_try_attack()` — Kiểm tra `spritecollide` liên tục trong khi đang lao.
- `enemy_update(player)` — Override: không dùng AI chase, chỉ lao theo `charge_direction`.

B.15 Tổng kết DSA trong Project

Bảng dưới đây tổng hợp tất cả thuật toán và cấu trúc dữ liệu được triển khai trong *Cycle of Curse*.

Thuật toán / CTDL	File triển khai	Mô tả & Độ phức tạp
BFS (Breadth-First Search)	enemy.py, summon.py	Pathfinding qua lưới 24×16 ô. $O(384)/\text{lần}$.
Singly Linked List	player.py, magic.py	<code>GhostNode</code> — <code>afterimage</code> khi dash/slash. $O(k)$ update.
Y-Sort	level.py	Sắp xếp sprites theo Y mỗi frame — depth rendering 2.5D.
FSM	player.py, enemy.py, summon.py	Trạng thái thực thể với luồng chuyển đổi ưu tiên rõ ràng.
Flood Fill (Stack DFS)	support.py	Xóa nền sprite từ 4 góc ảnh. $O(W \times H)$.
DAG Prerequisite Check	player.py, ui.py	Kiểm tra dependency graph trong Skill Tree trước khi unlock.
Spatial Grid Mapping	enemy.py	Chuyển tọa độ pixel $\rightarrow$ ô lưới (col, row) cho BFS.
Separation (Repulsion)	summon.py	Đẩy ngược summon chồng nhau trong <code>_orbit_owner</code> . $O(k^2)$.